COGNIZANT

Agentic AI Design Patterns

A practical guide to the core building blocks of modern AI systems

---

Author: **Cognizant**

Agentic AI Design Patterns

A practical guide to the core building blocks of modern AI systems — chaining, routing, parallelization, orchestration, and evaluation. These patterns are guidelines, not rigid rules. They're meant to inspire architectural thinking, not prescribe exact implementations. AI engineering patterns are less mature than classical software design patterns, and real-world systems almost always combine several of them.

1

Chaining

Sequential LLM steps

2

Routing

Classify & dispatch

3

Orchestration

Dynamic decomposition

4

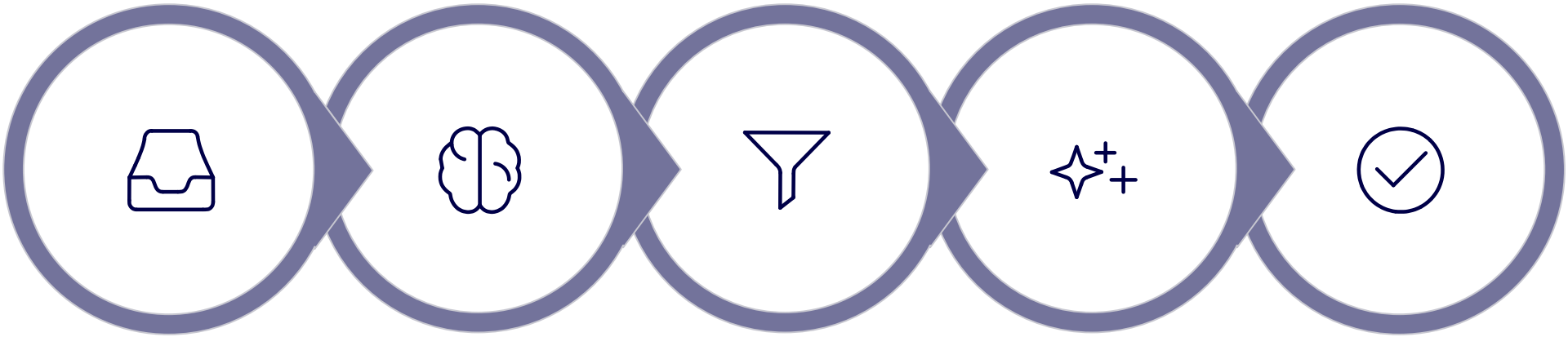
Evaluation

LLM as Judge

PATTERN 1

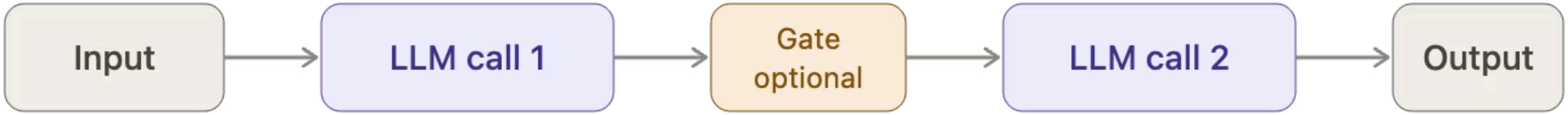
Prompt Chaining

Break a large, complex task into a **sequence of smaller LLM-driven subtasks**. The output of one LLM call becomes the structured input to the next — creating a pipeline of reasoning steps. Optional decision gates between steps can alter the flow based on intermediate results, enabling conditional branching without losing control.



Prompt chaining is one of the most common building blocks in production AI workflows. It improves task decomposition, makes outputs easier to debug, and allows each step to be optimized independently.

Prompt Chaining



Each step's output becomes the next step's input

PATTERN 2

Routing

The Core Idea

A classifier LLM inspects the incoming request and routes it to the most appropriate specialized workflow or downstream LLM — enabling clean separation of concerns.

Each specialized path focuses on a single domain, avoiding the trap of overloading a single prompt with all possible knowledge, instructions, and edge cases.

Example: Customer Support Router

🔧 Technical Support

Bugs, API errors, integration issues

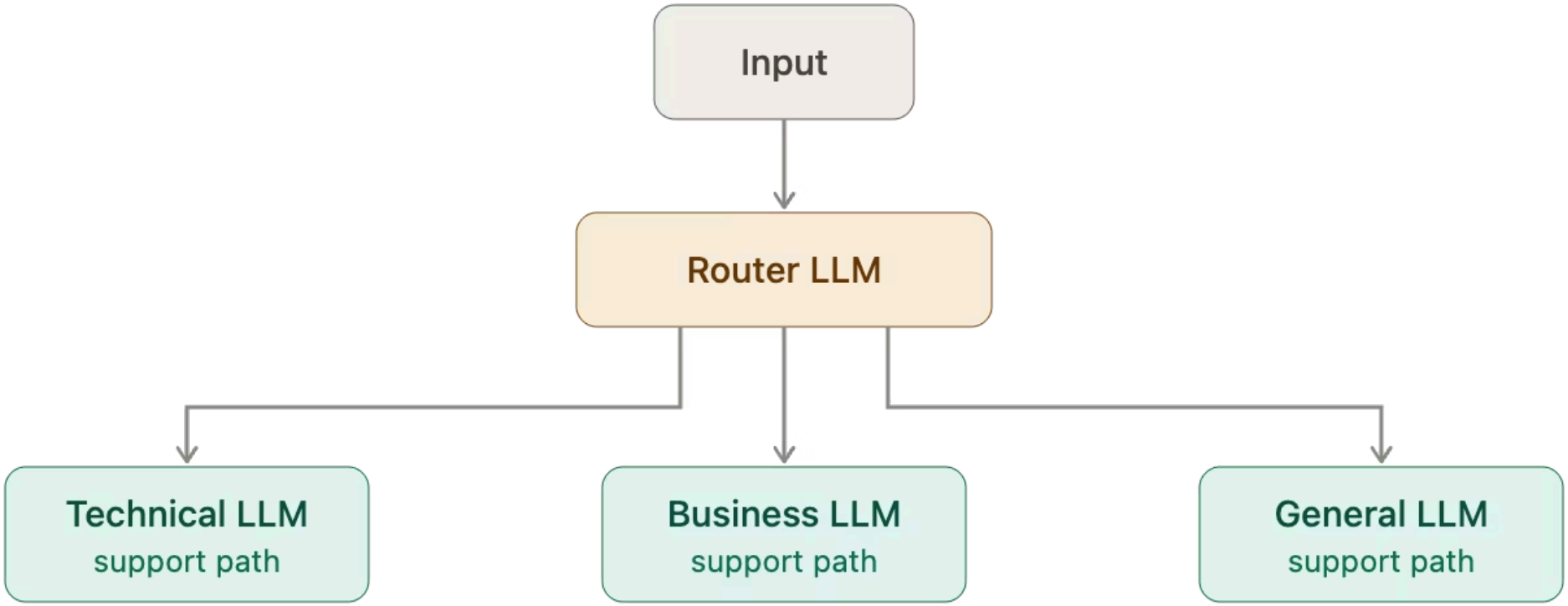
💼 Business Support

Billing, contracts, account management

💬 General Support

FAQs, product info, onboarding

Routing


Input
Router LLM
Technical LLM
support path
Business LLM
support path
General LLM
support path

PATTERN 3

Parallelization

When subtasks are independent of each other, there's no reason to run them sequentially. Parallelization splits a task into concurrent LLM calls, dramatically improving throughput and reducing latency. The **application code** — not an LLM — handles decomposition, parallel execution, and result aggregation.

Decompose

Application code splits the task into independent subtasks that share no dependencies.

Execute in Parallel

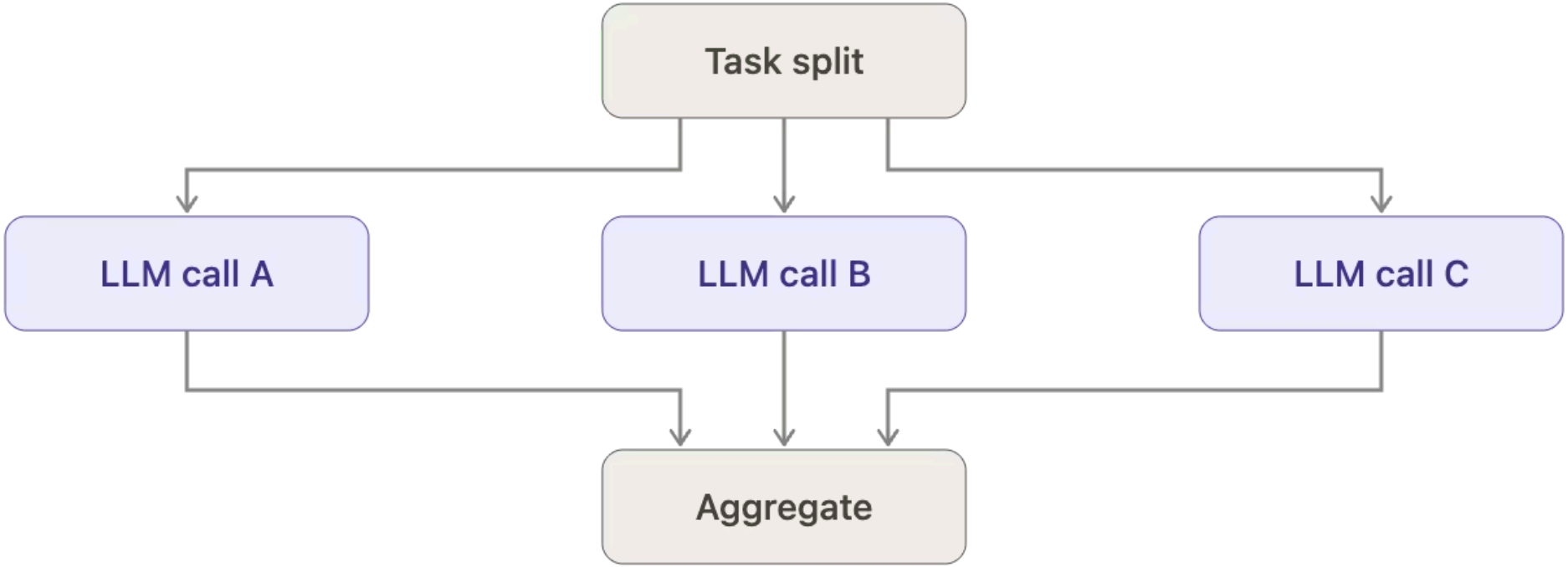
Multiple LLM calls run concurrently — each handling one subtask simultaneously.

Aggregate

Application code collects and merges the outputs into a single coherent result.

Key distinction: In parallelization, the **application code** drives all coordination logic — not an LLM. This makes it predictable and deterministic.

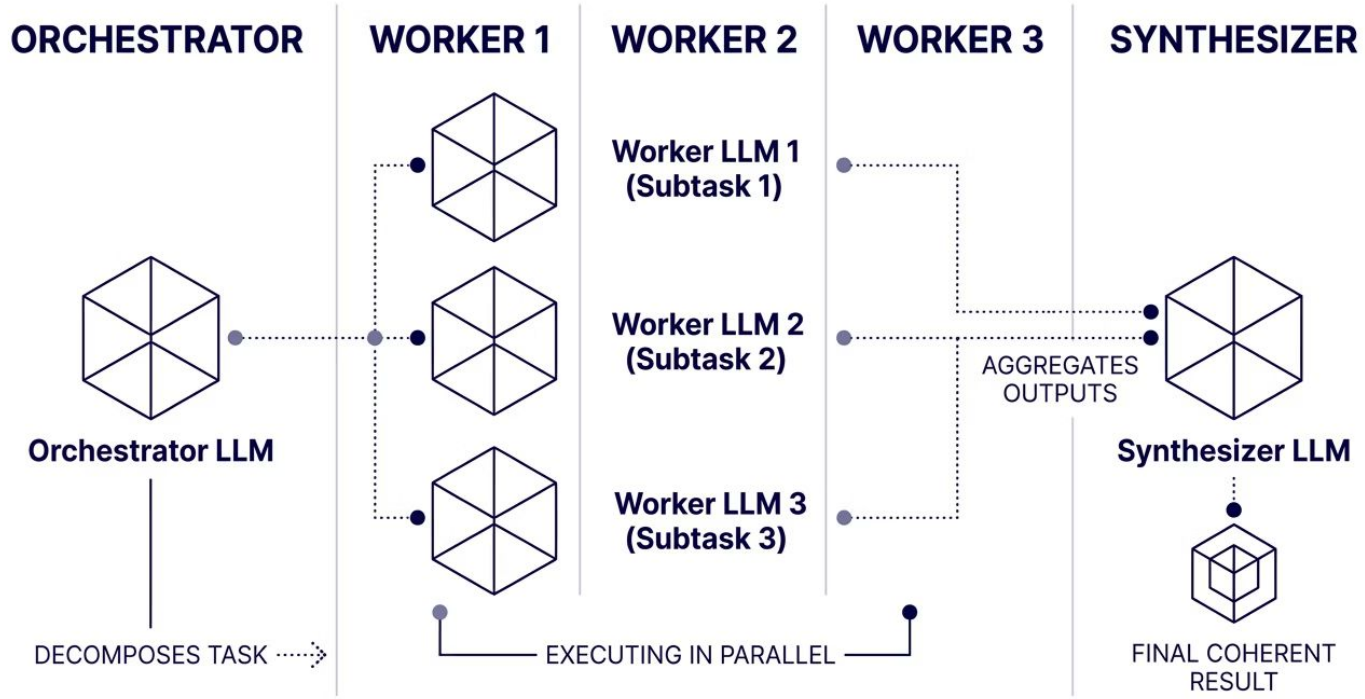
Parallelisation



PATTERN 4

Orchestrator-Worker Pattern

The most powerful evolution of parallelization: instead of application code deciding how to decompose a task, **an LLM orchestrator dynamically generates the decomposition plan** at runtime. This makes the system adaptive — able to handle tasks whose structure isn't known in advance.



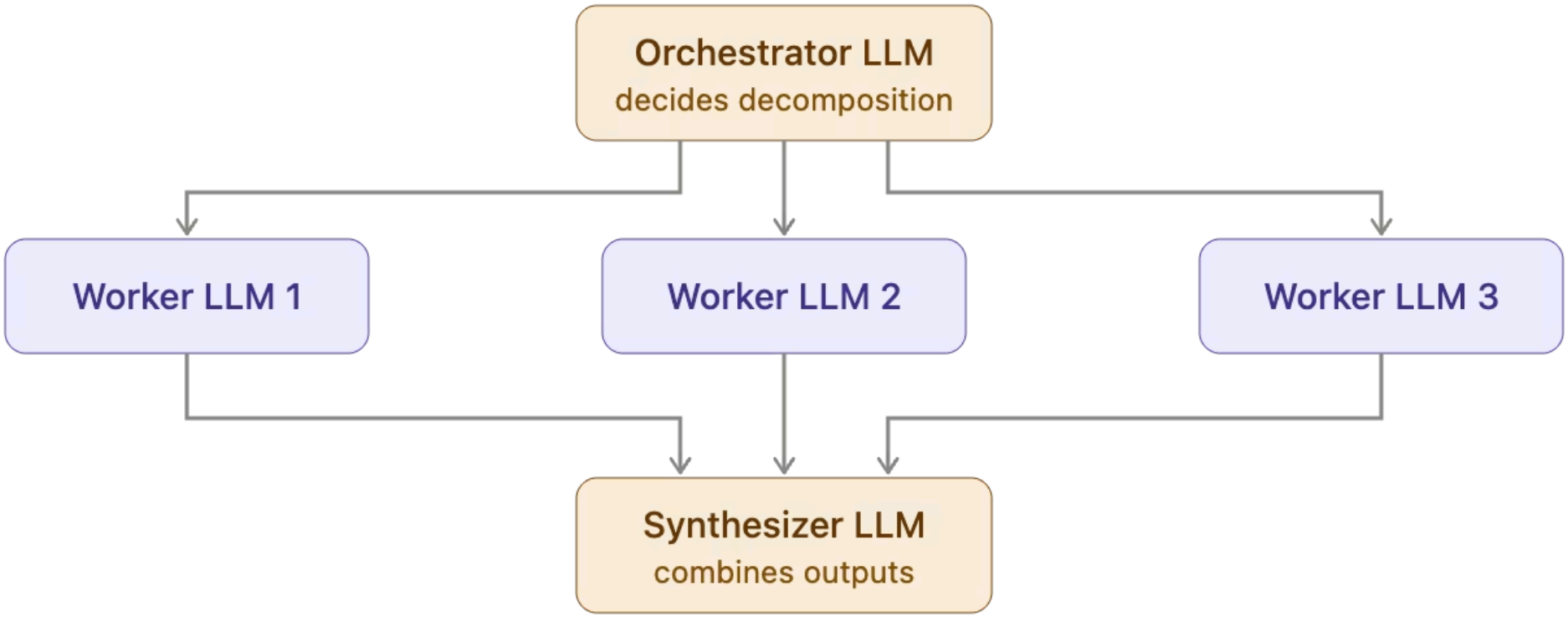
vs. Parallelization

In plain parallelization, application code controls decomposition and aggregation. In the Orchestrator-Worker pattern, **LLMs own that logic** — enabling flexible, goal-adaptive workflows.

Best Used When

The task structure is complex, variable, or can't be hardcoded. Common in research agents, multi-document synthesis, and adaptive code generation pipelines.

Orchestrator-Worker



Unlike parallelization, LLMs control decomposition and aggregation

Parallelization vs. Orchestrator-Worker

The core difference is who's making the decisions about splitting up and combining the work — **application code**, or **an LLM**.

⚙️ Parallelization

Your **application code** already knows how to split the task before anything runs

Example: you write code that says "run these 3 fixed subtasks in parallel" — the decomposition is hardcoded

Code also aggregates the results afterward (simple merge, concatenation, etc.)

The LLM's job is just to **execute each fixed subtask** — it has no say in how the work was divided or how results get combined

🧠 Orchestrator-Worker

An **LLM (the orchestrator)** decides how to break down the task, **dynamically**, based on the specific input

It doesn't know in advance how many subtasks there will be or what they are — it **figures that out at runtime**

Worker LLMs then execute whatever subtasks the orchestrator generated

Another **LLM (the synthesizer)** decides how to combine the worker outputs — also dynamic, not a fixed merge rule

Real-World Examples: Parallelization

The decomposition is fixed in code — same split every time.

🛡️ **Content Moderation Pipeline** — A social platform runs 5 LLM calls in parallel on every post: hate speech check, spam detection, misinformation flag, adult content filter, and copyright scan. The code always fires all 5 — no judgment needed about which to run.

📊 **Earnings Report Analysis** — A fintech tool splits a quarterly report into fixed sections (revenue, expenses, guidance, risks) and summarizes each in parallel. Every report gets the same 4-way split.

🌐 **Multilingual Translation** — A product description is sent simultaneously to 10 LLM calls, one per target language. The code knows exactly how many languages upfront.

Real-World Examples: Orchestrator-Worker

The LLM decides how to decompose — dynamically, based on the input.

🤖 **AI Software Engineer (e.g. Devin)** — Given "build a REST API for user auth", the orchestrator LLM breaks it into subtasks it invents on the fly: design schema, write endpoints, generate tests, write docs, check security. The number and nature of subtasks vary per request.

🔬 **Drug Discovery Research Agent** — A pharma AI receives "find candidates for target protein X". The orchestrator decides which databases to query, which assays to simulate, and how many worker agents to spin up — all based on what it learns as it goes.

📋 **Automated RFP Responder** — Given a complex procurement document, the orchestrator reads it first, identifies all unique requirements, assigns each to a specialist worker (legal, technical, pricing), then a synthesizer LLM assembles the final proposal.

A Concrete Example: Summarize a 50-Page Document

⚙️ Parallelization Approach

Your code applies a **fixed rule** you wrote in advance:

- 📄 Split document into **5 equal chunks** (pages 1–10, 11–20, etc.)
- 🤖 Run **5 LLM calls in parallel** — each summarizes its chunk
- 🔗 **Concatenate** the 5 summaries — your code does the merging

The chunking logic is **hardcoded**. Every document gets the same treatment regardless of its content or structure.

🧠 Orchestrator-Worker Approach

An **LLM reads the document first**, then decides how to split it:

- 🔍 Orchestrator LLM scans the document and **determines the natural structure**
- 🧩 It decides: **"3 workers"** — one for intro/background, one for methodology, one for results/conclusion
- 🤖 **Worker LLMs** execute each semantically meaningful subtask
- ✍️ A **synthesizer LLM** weaves the outputs into a coherent final summary

The decomposition is **dynamic** — it reflects *this* document's structure, not a generic rule.

The Rule of Thumb

> Rule of thumb: if you can write the decomposition logic yourself as a fixed piece of code, it's parallelization. If the decomposition itself requires judgment/reasoning that varies per input, you need an LLM doing that job — that's orchestrator-worker.

Parallelization — Fixed decomposition logic you write as code

Orchestrator-Worker — Dynamic decomposition requiring LLM judgment

PATTERN 5

Evaluator-Optimizer Pattern

Also known as **"LLM as a Judge"** — this pattern introduces a quality control loop into AI workflows. One LLM generates a solution; a separate evaluator LLM critiques it. If the output meets the bar, it's accepted. If not, the evaluator returns structured feedback and the generator tries again.

Generate

LLM produces an initial solution or response.



Evaluate

A second LLM scores or critiques the output against defined criteria.

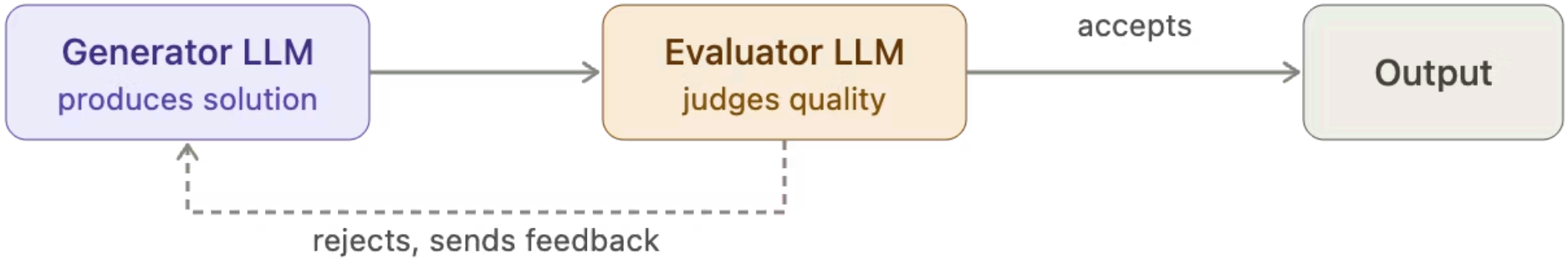
Accept

Once the evaluator is satisfied, the final output is returned.

Revise

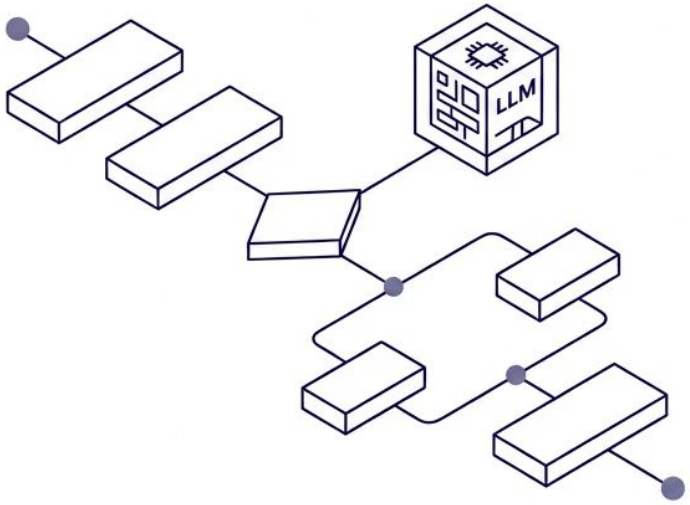
If rejected, the generator receives feedback and produces an improved version.

Evaluator Optimiser



Agents vs. Workflows

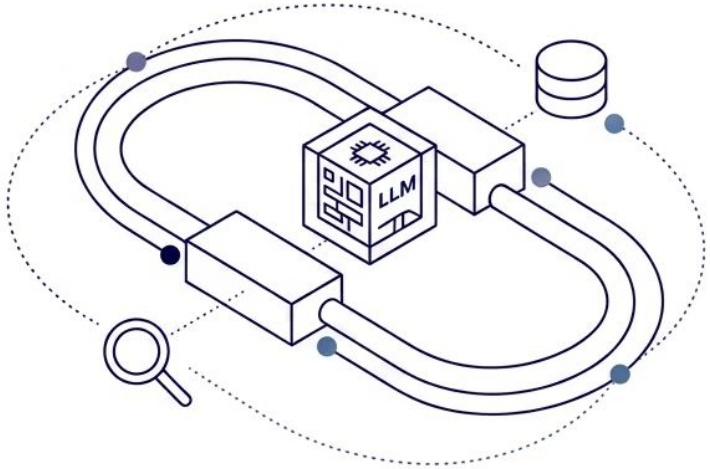
One of the most important distinctions in agentic AI architecture — and often the most misunderstood. The difference isn't about complexity; it's about **who controls the execution path**.



WORKFLOWS

LLM INFLUENCES WITHIN FIXED STRUCTURE.

- Deterministic, auditable paths for repeatable tasks.



AGENTS

LLM IN A LOOP WITH TOOLS TO REACH GOALS.

- Observes outcomes and continues acting.

Core agent mental model: An LLM in a loop — receiving input, interacting with tools and the environment, observing outcomes, and continuing to act until the goal is achieved.

How the Patterns Combine

Real production systems rarely use a single pattern in isolation. The power of these building blocks comes from composing them. Here's how they layer together in practice:



Prompt Chaining

Foundation: sequential steps with optional gates



+ Routing

Add intelligent dispatch to specialized sub-workflows



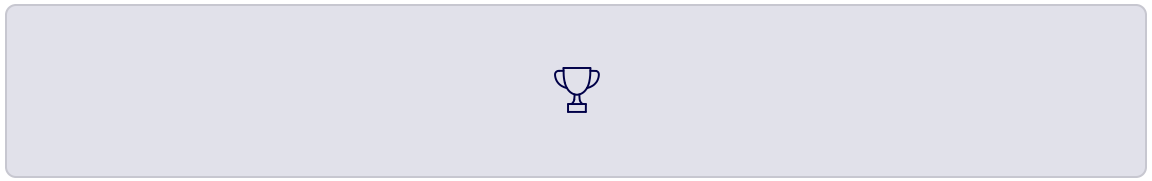
+ Parallelization

Speed up independent subtasks with concurrent execution



+ Orchestration

Let an LLM dynamically plan and delegate at runtime



+ Evaluation

Close the loop with LLM-driven quality control and refinement

Practical Takeaways

What matters most isn't memorizing pattern names — it's developing the architectural instinct to **recognize which building blocks apply and how to combine them** for your specific problem.

1	2	3
Patterns Are Guidelines AI engineering patterns are less mature than classical software design patterns. Treat them as mental models — inspiration for structure, not prescriptions for implementation.	**Real Systems Combine Patterns** Production AI systems routinely blend chaining, routing, orchestration, and evaluation. Learning to recognize these elements in the wild is the core skill.	**Workflows vs. Agents Is the Key Distinction** Understanding whether execution follows a predefined path or a goal-driven loop matters more for architecture decisions than any individual pattern.

The Building Blocks at a Glance

Every pattern covered in this module is a recurring element in production AI engineering. Keep these in your toolkit.

⛓️ Chaining

Sequential LLM subtasks with optional decision gates

🔀 Routing

Classify input and dispatch to specialized workflows

⚡ Parallelization

Concurrent LLM calls for independent subtasks

🎼 Orchestration

LLM-driven dynamic decomposition and synthesis

🏛️ Evaluation

LLM as Judge — generate, critique, revise, accept

🤖 Agents

LLM in a loop with tools, pursuing a goal

> The best AI architects don't memorize patterns — they develop the instinct to *recognize and compose* them under real constraints.